10-MINUTE PAPER REVIEW

Heterogeneous Model Serving at Different Granularities

ThunderServe and Hetis

Yu-Jui Liang
Advisor: Hsiang-Yun Cheng
National Taiwan University | Academia Sinica
28 Oct 2026

ThunderServe: Youhe Jiang et al., MLSys 2025
Hetis: Zizhao Mo et al., SC 2025

Presentation Roadmap

28 OCT / PLAN

1. **ThunderServe + Hetis**
   10 minutes. Review and comparison of last week's papers. This deck.
2. **Helix**
   10 minutes. Concise report on a new paper.
3. **Tessera**
   10 minutes. Concise report on a new paper.
4. **Two HBF papers**
   Extra 10 minutes. Backup, not counted in the formal 30 minutes.

Formal report: three blocks, 30 minutes. This deck is the first block.

ThunderServe: Why Heterogeneous GPU Serving Needs Phase-Aware Scheduling

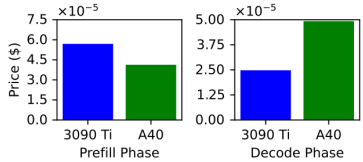


Figure 1. Prefill and decode prices for a single request with input and output lengths of 512 and 16 on 3090Ti and A40.

Observation

- Prefill: compute-intensive.
- Decode: memory-bandwidth-intensive.

A40: 149.7 FP16 TFLOPS. 3090 Ti: 1008 GB/s. Table 1.

Challenge

- Heterogeneous GPU resources.
- Limited and heterogeneous network bandwidth.
- KV cache transfer overhead.

Research question
How to jointly optimize GPU allocation, phase placement, and communication?

The better GPU can differ by phase. KV transfer makes a FLOPS-only or bandwidth-only split insufficient.

Source: ThunderServe Sec. 1, Fig. 1, Table 1. Prices in the figure are per request.

Joint Optimization of GPU Placement, Parallelism, and Routing

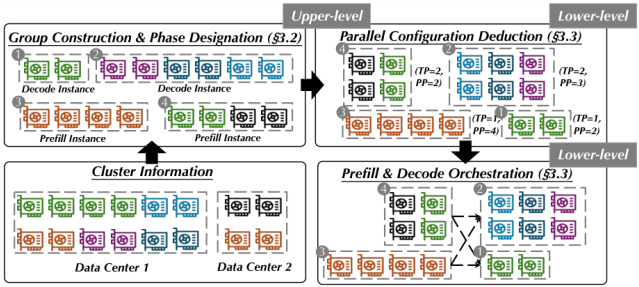


Figure 3. Workflow of our scheduling algorithm.

UPPER-LEVEL — TABU SEARCH

- GPU group construction.
- Prefill / decode phase designation.

LOWER-LEVEL — CONFIGURATION

- TP / PP configuration.
- Prefill / decode request routing.
- Network-aware KV cache transfer.

Upper level assigns groups and phases. Lower level sets parallelism and routing. Together they score one deployment plan.

Source: ThunderServe Sec. 3.2–3.4, Fig. 3.

Does Phase-Aware Scheduling Improve Serving Efficiency?

EXPERIMENT 1 — TABLE 3

Workload-aware deployment

Coding: **8 prefill / 4 decode**
Conversation: **4 prefill / 8 decode**

LLaMA-30B, same 32-GPU cloud pool. Coding has longer prompts; conversation has longer outputs.

EXPERIMENT 2 — FIGURE 9

	Coding	Conv.
vs. HexGen	1.5×	1.3×
vs. DistServe	1.5×	2.1×

HexGen: heterogeneous cloud GPUs.
DistServe: homogeneous 8×A100, similar price. Not the same machines.

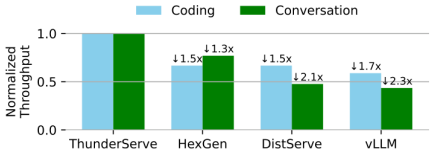


Figure 9. Throughput scaled by ThunderServe.

vLLM, the fourth group, is also in-house. HexGen and DistServe are different hardware setups.

Workload-aware placement improves overall serving throughput under heterogeneous resource constraints.

Source: ThunderServe Sec. 5.1–5.3, Fig. 9, Table 3. Relative throughput is ThunderServe over each baseline.

Phase-Aware Deployment Improves Cost Efficiency

1. JOINT OPTIMIZATION

One deployment plan

GPU grouping, phase designation, TP/PP, and request routing.

2. PERFORMANCE

1.7× average, up to 2.1×

Overall throughput claim in the paper. Not one bar of Figure 9.

ThunderServe demonstrates that heterogeneous GPU serving benefits from jointly optimizing inference phases, hardware allocation, and communication.

ThunderServe focuses on phase-level resource allocation; finer-grained workload partitioning is another possible direction.

Source: Abstract and Sec. 6 for the 1.7×/2.1× claim.

Hetis: Why Static Parallelism Wastes GPU Resources

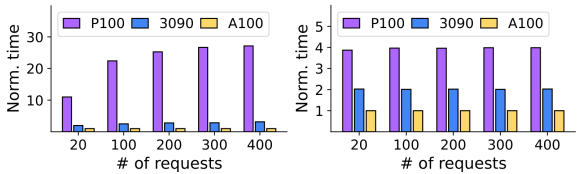


Figure 2: The execution time of the decoding MLP and Attention of one layer in Llama-70B across different GPUs. Sequence length of each request is set to 1000.

Why one parallelism for every module?

- MLP and decode attention do not show the same GPU gap.
- A low-end GPU can drag dense computation.
- A static split can also leave KV-cache space idle.

Fig. 2 is one Llama-70B decode layer, sequence length 1000. Not end-to-end latency, and not a bandwidth chart. The paper asks for module-aware, fine-grained, dynamic parallelism.

GPU performance heterogeneity is module-dependent; static model partitioning cannot efficiently balance both computation and memory.

Source: Hetis Sec. 2.2–2.4, Figs. 1–2. Text: P100 lags A100 by up to 40.4× on average in decode MLP.

Hetis: Selective Primary Workers and Dynamic Head-wise Attention

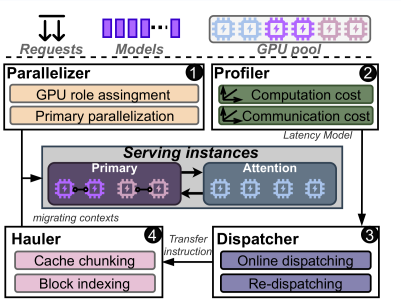


Figure 3: The system architecture of Hetis.

PRIMARY WORKERS

Parallelizer searches dense-compute plus communication cost. They jointly hold the model with TP/PP, and DP when needed. They run prefill, decode dense modules, and heads that stay local.

ATTENTION WORKERS

GPUs that add little to dense compute. They run some decode attention and its KV cache, without a second full copy of the weights. Default exclusion threshold $\Delta = 0.05$. Selection is cost, not a FLOPS or bandwidth ranking.

PER-LAYER DECODE TRAFFIC

Primary computes QKV. For an offloaded head it sends that token's Q/K/V; the worker attends with its stored KV cache and returns the output. Full history is not resent each step. GQA keeps a query group with its KV heads.

Hetis concentrates dense computation on suitable GPUs while dynamically distributing decode attention at head granularity.

Source: Hetis Sec. 3–5, Fig. 3. Fig. 3 also names Profiler, Dispatcher, and Hauler.

Hetis: Does Head-wise Parallelism Improve Serving Performance?

Q1 — END-TO-END SERVING

Fig. 10 y-axis is latency (s/token), not throughput.

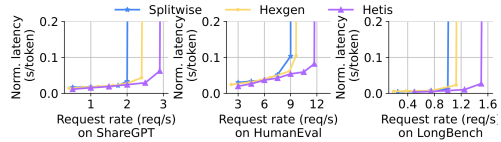


Figure 10: The normalized end-to-end latency across different datasets under Llama-70B.

Up to 2.25× throughput vs. Splitwise, and 1.33× vs. Hexgen. Maxima across settings, not averages. On this Llama-70B run, A100 and 3090 were primaries and P100s were attention workers. A search result, not a ranking.

Q2 — KV CACHE

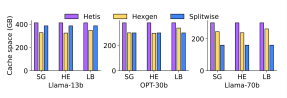


Figure 11: The maximum available KV Cache space during inference across models and datasets.

Up to 1.87× available KV cache versus the tested baselines.

Fine-grained attention placement improves serving capacity by reducing compute imbalance and KV cache underutilization.

Source: Hetis Sec. 7.1–7.3, Figs. 8–13. The 1.49× figure is decode-attention module latency, not end-to-end latency.

Hetis: Main Contributions and Takeaways

1

Module-aware workers

Assign GPUs by what each module costs, not by one static split of the whole model.

2

Head-level attention

Offload decode attention by heads. Unselected heads stay on the primary.

3

Online adaptation

Dispatcher reallocates heads. Hauler migrates KV cache at head granularity.

Implementation: vLLM extension.

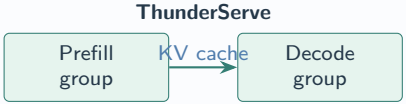
Head-level KV-cache blocks, custom CUDA kernels, NCCL peer-to-peer, and reuse of PagedAttention. Base framework, not a Hetis repository: `https://github.com/vllm-project/vllm`

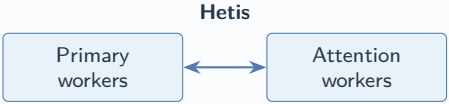
End-to-end results use the real GPU cluster. Sec. 7.4 adds a large-scale simulation of parallelizer search cost.

Hetis improves heterogeneous GPU utilization through module-aware, fine-grained, and dynamic parallelism.

Source: Hetis Sec. 6 and Sec. 9 for the system; Sec. 7.1–7.2 for the cluster results; Sec. 7.4 for the search simulation.

ThunderServe vs. Hetis: Different Granularities of GPU Specialization





	ThunderServe	Hetis
Objective	Cost-efficient serving under workload and network limits	Less compute imbalance and better memory use
Granularity	Prefill / decode phases	Dense modules / attention heads
Weights	Each phase replica hosts weights; TP/PP allowed	Primaries hold the model. Attention workers do not
Communication	KV cache between prefill and decode replicas	Current-token Q/K/V and outputs for offloaded heads
Scheduling	Grouping, phase, TP/PP, routing, lightweight rescheduling	Primary selection, online head dispatch, KV migration

ThunderServe optimizes phase-level deployment; Hetis optimizes module-aware execution and head-level attention placement.

Source: Diagrams are conceptual, not paper figures. Do not rank the papers by peak throughput: hardware, workloads, and baselines differ.

10-MINUTE PAPER REVIEW

Thank you

ThunderServe and Hetis

Yu-Jui Liang

28 Oct 2026

Backup Topics

Q&A / 21 OCT DECKS

THUNDERSERVE

Phase and network — 2 slides

GPU phase characteristics and network heterogeneity.

Hierarchical scheduling — 2 slides

Two-level search and Tabu Search operations.

TP / PP configuration — 1 slide

Parallel configuration for each GPU group.

Routing and KV transfer — 4 slides

Replica routing, KV cost, and network-aware placement.

Lightweight rescheduling — 1 slide

Phase flip and routing update without a full search.

KV cache compression — 1 slide

4-bit KV for communication only.

Throughput and deployment — 2 slides

End-to-end throughput and Table 3 plans.

Simulator accuracy — 1 slide

Simulator and alpha-beta communication model.

HETIS

Motivation — 2 slides

KV underutilization and module-dependent GPU gaps.

Workers and architecture — 2 slides

Primary workers, attention workers, and the system.

Head-wise attention — 3 slides

Head split, dynamic allocation, and GQA constraints.

Dispatch and migration — 3 slides

Re-dispatching, head-level KV, and live migration.

Llama-70B results — 2 slides

End-to-end results and usable KV cache capacity.

HBM / HBF implications — 1 slide

Follow-up questions for memory co-design. Not a result.

Return to Review

These pages support questions. They are outside the 10-minute talk, and the formal footer stays $n/12$.

Different GPUs Favor Different Inference Phases

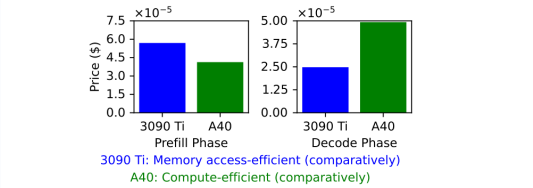


Figure 1. Prefill and decode prices for a single request with input and output lengths of 512 and 16 on 3090Ti and A40.

Fig. 1: Per-request cost; input 512 / output 16 tokens

- A40 has lower prefill cost in this experiment.
- Its compute capability favors prompt processing.
- 3090 Ti has lower decode cost in this experiment.
- Its higher memory bandwidth favors token generation.

Choose GPUs by phase, workload and rental cost, rather than a single GPU ranking.

Cloud Networking Is Highly Heterogeneous

THUNDERSERVE / BACKGROUND & MOTIVATION

- Left: 32 cloud GPUs have uneven connectivity across GPU pairs.
- Right: 8 in-house A100 GPUs have much faster links; color scales differ.
- Phase splitting adds a transfer of prompt KV cache between replicas.
- Low-bandwidth paths can erase the benefit of specializing GPUs by phase.

Deployment must account for both GPU capability and the communication topology.

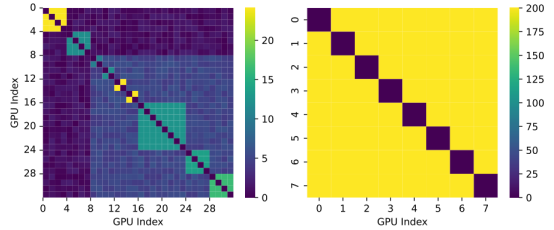


Figure 13. Heat map of inter-connection bandwidth matrix in the cloud (left) and in-house (right) settings.

Fig. 13: Pairwise bandwidth measured with NCCL

Scheduling as Two-Level Hierarchical Optimization

THUNDERSERVE / METHODOLOGY

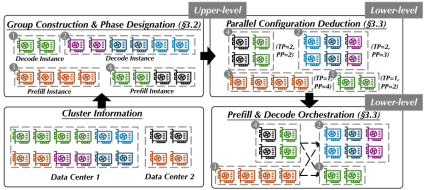


Figure 3. Workflow of our scheduling algorithm.

Fig. 3: Workflow of the scheduling algorithm

- Upper-level optimizes group construction and phase designation.
- Lower-level optimizes parallel configuration and replica orchestration.
- Given a candidate upper-level solution, the lower-level returns its SLO attainment.
- The upper-level search uses that performance feedback to improve the deployment plan.

Objective: maximize the fraction of requests meeting the end-to-end latency deadline.

Four Neighbor Operations in Tabu Search

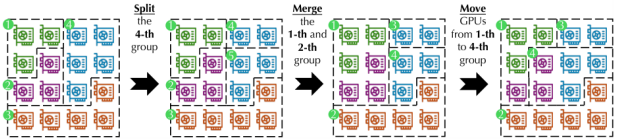


Figure 4. Examples of neighbor construction in tabu search (changes in phase designation are omitted for simplicity).

Fig. 4: Neighbor construction in tabu search

- **Flip:** change a serving group's prefill or decode phase.
- **Split:** divide one GPU group into two.
- **Merge:** combine two GPU groups.
- **Move:** transfer GPUs of a given type between groups.

Tabu search explores changes in both GPU grouping and phase designation.

Deducing Parallel Configurations for Each Group

THUNDERSERVE / METHODOLOGY

TENSOR MODEL PARALLELISM

TP: split work within layers

Disallow cross-node TP; Appendix B additionally limits TP to GPUs of a single type.

PIPELINE ROUTING

Dynamic programming

Identify pipeline communication paths that minimize cross-stage communication cost.

PIPELINE MODEL PARALLELISM

PP: split layers into stages

Account for different GPU memory capacities and computing abilities.

PARALLEL CONFIGURATION

Phase-specific objective

Select the lowest predicted latency for prefill; highest predicted throughput for decode among candidates.

Deduce each group's parallel configuration before prefill / decode orchestration.

Source: Section 3.3 and Appendix B; cost model from HexGen.

Orchestrating Prefill and Decode Replicas

THUNDERSERVE / METHODOLOGY

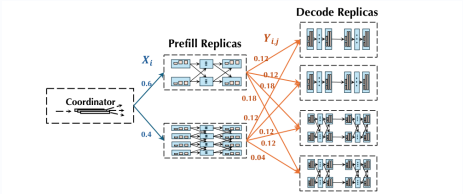


Figure 5. An example orchestration of prefill and decode replicas.

Fig. 5: Example prefill-to-decode orchestration

- A request executes on a prefill replica and then a decode replica.
- KV cache must be transmitted between the two replicas.
- DistServe's inference task simulator estimates pairwise SLO attainment.
- The resulting matrix D_{ij} scores each prefill–decode pair, including KV transfer cost.

Request routing must account for heterogeneous inter-replica communication.

Modeling KV Cache Communication Cost

THUNDERSERVE / METHODOLOGY

Paper Eq. (1): alpha-beta communication model

$$T_{ij}^{(\mathrm{kv\ comm})} = \alpha_{ij} + \frac{2bshN_{\mathrm{bytes}}}{\beta_{ij}}$$

b: batch size; s: sequence length; h: block hidden size; N_{bytes}: bytes per KV element.
α_{ij}: startup latency; β_{ij}: bandwidth in bytes/s; factor 2: keys and values.

- Transfer time grows with payload size and falls with link bandwidth.
- **Scope note:** the printed equation has no layer-count term.
- For full-model KV cache, tensor accounting gives $V_{\mathrm{KV}} = 2Lbsh_{\mathrm{KV}}q$ bytes before metadata (L: layers; h_{KV}: total KV-head width; q: bytes/element).

Account for transferred layers, KV heads and sharding when applying the model.

Source: Section 3.3, Eq. (1). Full-model tensor accounting is an explanatory clarification.

Network-Aware Deployment Plans

THUNDERSERVE / METHODOLOGY

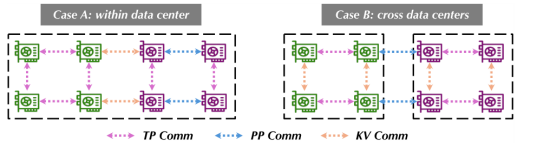


Figure 17. ThunderServe deployment plans on different cases.

Fig. 17; Appendix H: LLaMA-30B, 4 A40 + 4 3090 Ti

- At 40 Gbps, use 4-A40 for prefill and 4-3090Ti for decode.
- Throughput: 3292 vs. 1610 tokens/s for the non-disaggregating baseline.
- At 5 Gbps, both phases use 2-A40 and 2-3090Ti across instances.
- 2196 tokens/s (1.4× baseline): KV stays within instances; PP crosses instances.

The scheduler changes deployment as inter-instance bandwidth changes.

Request-Time Breakdown in Table 5

THUNDERSERVE / METHODOLOGY

LLaMA-30B, 4×A40 + 4×3090Ti. High / low are the 40 Gbps and 5 Gbps deployments on the previous slide.

Configuration	Prefill	KV comm	Decode	Throughput
Baseline	884 ms	0 ms	1689 ms	1610 tok/s
ThunderServe, 40 Gbps	698 ms	133 ms	1126 ms	3292 tok/s
ThunderServe, 5 Gbps	964 ms	41 ms	1846 ms	2196 tok/s

The 5 Gbps row has less KV time because the deployment stops sending KV across instances, not because the slow link is faster.

Do not invert one request's prefill + decode time and call that system throughput.

Source: Appendix H, Table 5.

Lightweight Re-scheduling in Real Time

THUNDERSERVE / METHODOLOGY

WHY RE-SCHEDULE?

Workload variability

Workload characteristics and GPU availability change in online cloud services.

PARALLELISM

Skip new deduction

Keep surviving group construction and existing TP / PP configurations.

TABU SEARCH

Only flipping phase designation

Do not use splitting, merging or moving groups during lightweight re-scheduling.

ORCHESTRATION

Solve routing again

Optimize orchestration with updated phase designation; no model-parameter reloading.

Keeping weights in place saves reload time, but restricts the plans that can be explored.

Source: Section 3.4. Quantitative overhead appears in Table 4.

Quantize KV Cache for Communication Only

THUNDERSERVE / METHODOLOGY

- Quantize and pack 16-bit KV values into 4-bit elements for transfer.
- Decode replica unpacks and dequantizes received KV cache.
- Both prefill and decode computations use 16-bit KV cache.
- LLaMA-7B, two A5000s, 40 Gbps: communication share falls from 16–30% to 4–9%.

Dequantization restores the 16-bit representation, not the information lost in quantization.

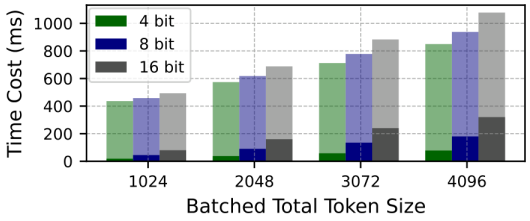


Figure 18. Impact of KV cache communication compression. (Non-transparent: time cost of KV cache communication. Transparent: end-to-end processing time.)

Fig. 18: Impact of KV cache communication compression

Return to Review | Backup Topics

End-to-End System Throughput

THUNDERSERVE / EXPERIMENTAL RESULTS

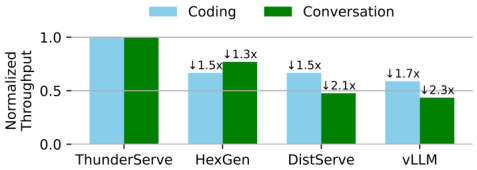


Figure 9. Throughput scaled by ThunderServe.

Fig. 9: Throughput scaled by ThunderServe

- Against HexGen: 1.5× (coding) and 1.3× (conversation) throughput.
- Against DistServe: 1.5× (coding) and 2.1× (conversation).
- Fig. 9 also reports 1.7× / 2.3× versus vLLM (coding / conversation).
- Fig. 9 normalizes ThunderServe to 1.0; its vLLM 2.3× exceeds the abstract's stated 2.1× maximum.

Use baseline-specific ratios; the abstract and Fig. 9 do not state a consistent overall maximum.

Full Deployment Plans from Table 3

THUNDERSERVE / EXPERIMENTAL RESULTS

Workload	GPUs	Strategy	Replicas
Coding	8×A40	TP=2, PP=1	4 prefill
	4×A5000	TP=4, PP=1	1 prefill
	4×A6000	TP=2, PP=1	2 prefill
	2×A5000+2×3090Ti	TP=2, PP=2	1 prefill
	4×3090Ti	TP=2, PP=2	1 decode
	4×A6000	TP=1, PP=2	2 decode
	2×A5000+2×3090Ti	TP=2, PP=2	1 decode
Conversation	6×A40	TP=2, PP=1	3 prefill
	2×A5000+2×3090Ti	TP=2, PP=2	1 prefill
	4×3090Ti	TP=2, PP=2	1 decode
	2×A40	TP=1, PP=2	1 decode
	4×A5000	TP=2, PP=2	1 decode
	8×A6000	TP=1, PP=2	4 decode
	2×A5000+2×3090Ti	TP=2, PP=2	1 decode

Coding is 8 prefill + 4 decode; conversation is 4 prefill + 8 decode. Both use 12 replicas.

Source: Table 3 and Appendix F. The previous slide highlighted only the A40 and one mixed row.

Simulator and Alpha-Beta Model Accuracy

THUNDERSERVE / EXPERIMENTAL RESULTS

- Appendix J uses LLaMA-30B micro-benchmarks.
- Vary SLO scales and batched token sizes.
- Left: simulated attainment differs from measurements by at most 2 percentage points in the plotted cases.
- Right: alpha-beta latency differs by at most 2 ms across the four plotted token sizes.

These micro-benchmarks support the estimates in the tested settings; they are not a general error bound.

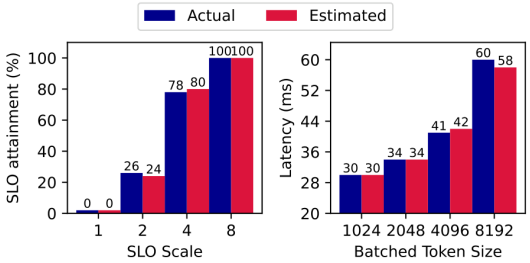
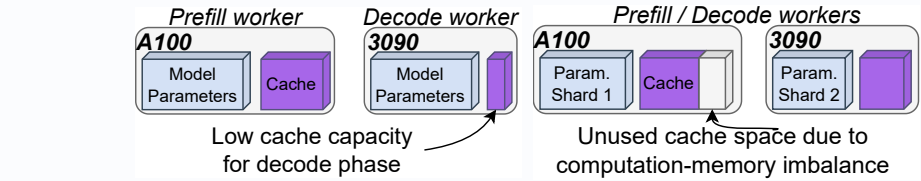


Figure 19. Comparison of benchmarked and estimated performance metrics for simulator (left) and alpha-beta model (right).

Fig. 19: Benchmarked vs. estimated performance

Return to Review | Backup Topics

Coarse Splitting Wastes KV Cache Capacity



(a) Phase-splitting **(b) Parameter-splitting**

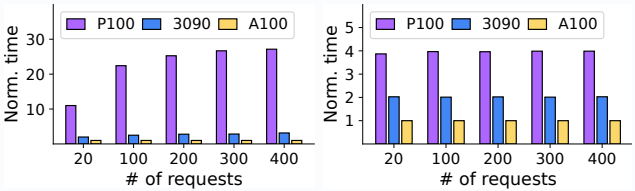
Fig. 1; Section 2.3: Two sources of memory inefficiency

- Phase splitting stores separate model copies and confines decode KV cache to decode workers.
- In the FP16 7B example, a 24 GB 3090 has only about 10 GB left for KV cache.
- Parameter splitting balances compute, but fixes where each request stores its KV cache.
- One GPU can run out of cache while another still has free memory.

Balancing computation alone does not guarantee usable KV cache capacity.

Next | Return to Review | Backup Topics

MLP and Attention Scale Differently



(a) MLP **(b) Attention**

Fig. 2: One Llama-70B decode layer; context length 1000

- The P100/A100 execution-time gap is much larger for MLP than for attention.
- Adding slow GPUs to dense operations can cost more communication than it saves in compute.
- Decode attention has a smaller performance gap and can use otherwise idle GPU memory.
- The preferred GPU partition can differ across modules within the same layer.

Use selected GPUs for dense work and a wider pool for decode attention.

Previous | Return to Review | Backup Topics

Primary Workers and Attention Workers

HETIS / METHODOLOGY

Phase	Operation	Primary workers	Attention workers
Prefill	QKV / Output Projections, MLP	Execute	—
Prefill	Attention Core & KV Cache Creation	Execute	—
Decode	QKV / Output Projections, MLP	Execute	—
Decode	Attention Core	Process assigned heads	Process offloaded heads
Decode	KV Cache Storage	Store assigned heads	Store offloaded heads

$$\text{Attention core}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_h})V$$

Here, the attention core is parameter-free *after* Q, K and V are produced; the projections still require model weights.

Primary workers execute both prefill and decode, while attention workers only assist with decode attention and KV cache storage.

Source: Sections 2.4, 3.2 and 4. Attention core terminology clarified for this presentation.

Hetis System Architecture

- Parallelizer chooses Primary GPUs and their DP / PP / TP configuration.
- Profiler estimates GPU computation and communication cost.
- Dispatcher assigns heads to Primary and Attention workers, and selectively re-dispatches existing requests.
- Hauler manages head-level KV blocks and migrates the required cache shards.

Offline placement and profiling support online head allocation.

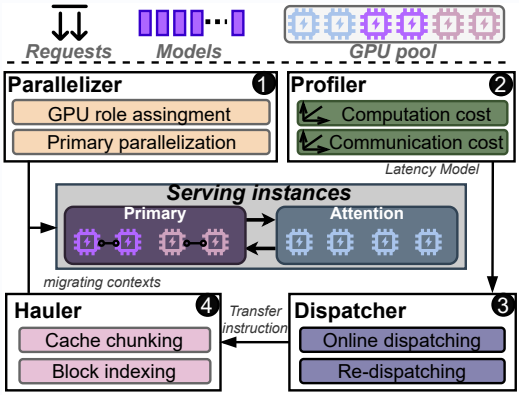
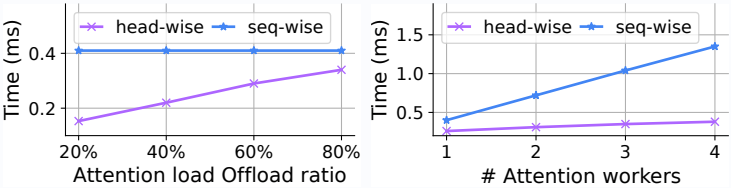


Fig. 3: Parallelizer, Profiler, Dispatcher and Hauler

Why Split Attention by Head?



(a) Overhead under various loads **(b) Overhead across workers**

Fig. 5: Llama-70B, 100 Gbps; communication overhead

- Batch-wise splitting moves whole requests, which is coarse when context lengths differ.
- Sequence-wise splitting replicates query vectors across workers holding different token ranges.
- Head-wise splitting transfers only the assigned head vectors and their results.
- Reported communication speedups over sequence splitting: 2.68× at 20% offload; up to 3.55× with four workers.

Independent heads reduce communication and allow finer cache placement.

Dynamic Attention Placement Across Requests

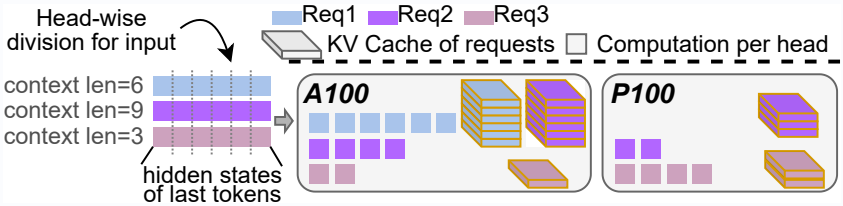


Fig. 6: Three requests, six heads each; context lengths 6, 9 and 3

- Each request can use a different head distribution across the same GPU pool.
- KV cache follows the assigned heads; longer contexts require more cache per head.
- Primary and Attention workers compute their assigned heads, then gather the head outputs.
- Re-dispatching changes selected assignments and migrates only the cache that must move.

Dynamic parallelism changes placement, not the number of model heads or the attention definition.

Head Integrity and Grouped-Query Attention

HETIS / METHODOLOGY

$$r = \frac{H_Q}{H_{KV}}, \qquad \sum_{i=1}^{N_k} x_i^j = H_Q, \qquad x_i^j / r \in \mathbb{Z}_{\geq 0}.$$

x_i^j: query heads of request j assigned to GPU i in pipeline stage k.

- MHA: $r = 1$; each query head has its own key/value head.
- GQA: each KV head is shared by a group of r query heads; Hetis preserves whole groups.
- Illustrative example: $H_Q = 8$, $H_{KV} = 2$, so $r = 4$; valid per-GPU allocations are 0, 4 or 8 query heads.
- Each assigned head uses its full context. Head-wise computation needs no cross-worker softmax reduction over sequence shards.

GQA reduces KV volume, but also makes the placement unit coarser than one query head.

Source: Sections 4.2, 5.2.1 and 6. Example is explanatory, not a measured model configuration.

Re-dispatching to Balance Computation

HETIS / METHODOLOGY

Ideal attention time over existing requests $\mathbb{S}(t)$:

$$f^* = \min \max_i f_i(\vec{x}_i(t)) \quad \text{s.t.} \quad \sum_{i=1}^{N_k} \sum_{j \in \mathbb{S}(t)} x_i^j(t)\, l_j(t) \leq \sum_{i=1}^{N_k} \frac{r \cdot \mathrm{M}_i}{2}.$$

- If the deviation from the current attention time exceeds Θ (default 50%), pick the slowest GPU and its heaviest request.
- Re-dispatch that request with Eq. (7). Keep reusable KV shards; transfer only changed assignments.
- The displayed constraint is one cluster-wide sum. The paper does not define how "deviation" is normalized.

Growing contexts can create stragglers even if initial head placement was balanced.

Source: Section 5.3.1. Head-integrity constraints are not restated in this f^* display.

Head-Level KV Management and Migration

HETIS / METHODOLOGY

CACHE LAYOUT

Split blocks along the head dimension

Extend vLLM's fixed-sized blocks; index by sequence ID, token position and head ID.

COMMUNICATION

NCCL point-to-point groups

Transfer head-specific inputs and results between Primary and Attention workers.

ATTENTION KERNEL

Reuse PagedAttention

Compute local heads independently; accelerate block indexing with CPU cores.

LIVE MIGRATION

Low-priority CUDA streams

Reduce interference with ongoing inference communication; bandwidth cost still exists.

Fine-grained placement requires matching cache indexing, kernels and transfer support.

Source: Section 6. Low-priority migration is not a guarantee of zero overhead.

Dynamic Head and Cache Allocation

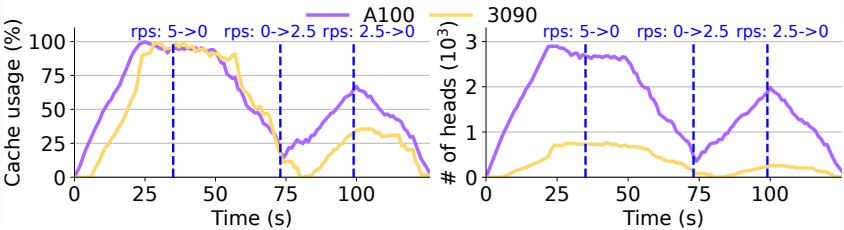


Fig. 14: Llama-13B; one A100 Primary + two 3090 Attention GPUs

- ShareGPT arrival rate changes over time: $5 \rightarrow 0 \rightarrow 2.5 \rightarrow 0$ req/s.
- A100 processes more heads, while both GPU types can reach high cache utilization.
- 3090 participation starts later under light load because remote execution adds communication cost.
- The head counts and cache usage change with both arrivals and completion of ongoing requests.

Hetis activates remote attention when its compute and memory benefits justify communication.

Llama-70B: GQA End-to-End Results

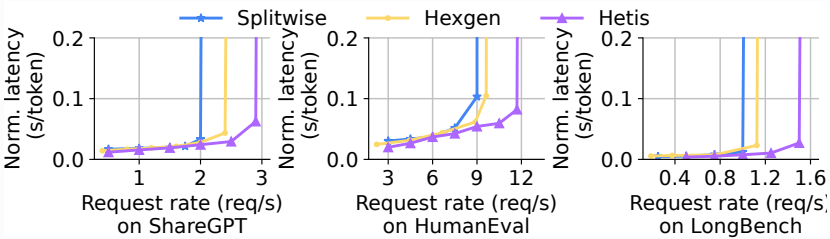


Fig. 10: Llama-70B; Section 7.2 summarizes Figures 8–10

- Hetis retains the advantage with a GQA model, whose KV cache per query head is smaller.
- A100 / 3090 handle dense work; P100 contributes decode attention and cache capacity.
- Across the end-to-end experiments, reported throughput gains are up to 2.25× vs. Splitwise and 1.33× vs. HexGen.
- These maxima summarize the experiments; they are not guaranteed for every panel or workload.

Module placement and dynamic attention sharing remain useful with GQA.

Maximum Usable KV Cache Space

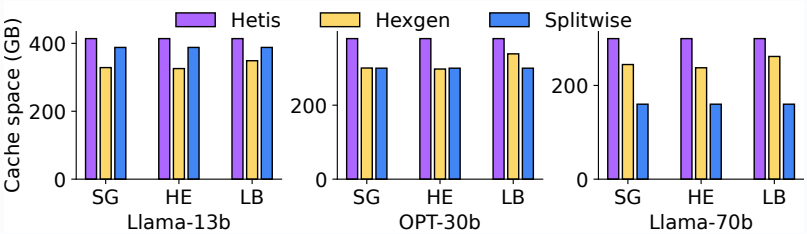


Fig. 11: Maximum available KV cache during inference

- Hetis reports up to 1.87× more usable KV cache capacity than the baselines.
- It avoids storing full model copies on attention-only workers.
- Dynamic allocation reduces the idle cache caused by fixed compute-based partitions.
- The gain is usable cache capacity, not additional physical GPU memory or a guaranteed equal throughput gain.

More usable cache allows the system to host more concurrent request state.

Previous | Return to Review | Backup Topics

Implications for GPU + HBM / HBF Co-design

CROSS-PAPER / RESEARCH DIRECTIONS

QUESTION 1

Memory-aware placement

How do HBM bandwidth and capacity alter phase or head assignment?

QUESTION 2

Tiered KV caches

Could a slower, larger tier improve capacity without hurting TPOT?

QUESTION 3

Serving simulation

What hardware-model extensions are needed in LLMServingSim?

These are follow-up research hypotheses, not evaluated HBF results in either paper.

Discussion focus: scheduling policy, cost-model fidelity, and memory hierarchy.